Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Session 2026-2027

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge,Problem analysis, Design/development of solutions,Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World,Ethics, Individual and Collaborative Team work,Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Sheikh Wasimuddin
Name of Student

Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Session	**2026-27 (ODD)**	**Course Name**	**PE-V : Lab : Blockchain Technology**
Semester	**7**	**Course Code**	**23IOT1748**
Roll No	**61**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**10**
Course Outcome	**1. Demonstrate basics of distributed systems and cryptographic primitives used in blockchain.** **2. Outline various research aspects for development of blockchain.** **3. Make use of different security protocols to build secure blockchain applications.** **4. Analyze smart contracts and Hyperledger Fabric used in blockchain. 5. Determine the performance of applications developed with Bitcoin and Ethereum blockchain.**
Aim	Develop and Deploy a Mini Decentralized Application (DApp) Using Solidity, Remix IDE, and MetaMask.
Problem Statement	Develop a blockchain-based DApp to securely track product warranties and ownership, enabling transparent and tamper-resistant verification.
Theory (100 words)	WarrantyChain is a blockchain-based decentralized application (DApp) designed to securely manage and verify product warranty and ownership information. Traditional warranty systems usually depend on centralized databases and paper documents, which can be lost, modified, or difficult to verify. Blockchain provides a decentralized and tamper-resistant solution where important product records can be stored securely and verified by authorized users. The system uses **Solidity smart contracts** to store product information such as product ID, product name, owner details, warranty period, and warranty expiry date. Once the information is recorded on the blockchain, it becomes transparent and difficult to modify without authorization. This helps reduce fraudulent warranty claims and provides a reliable history of product ownership. The DApp is deployed on the **Ethereum Sepolia Testnet** using **Remix IDE**. **MetaMask** acts as the digital wallet and connects users to the blockchain, allowing them to approve transactions and interact with the smart contract. Each transaction is recorded on the blockchain, providing transparency and traceability. A **React-based frontend** can be used to provide a simple user interface for

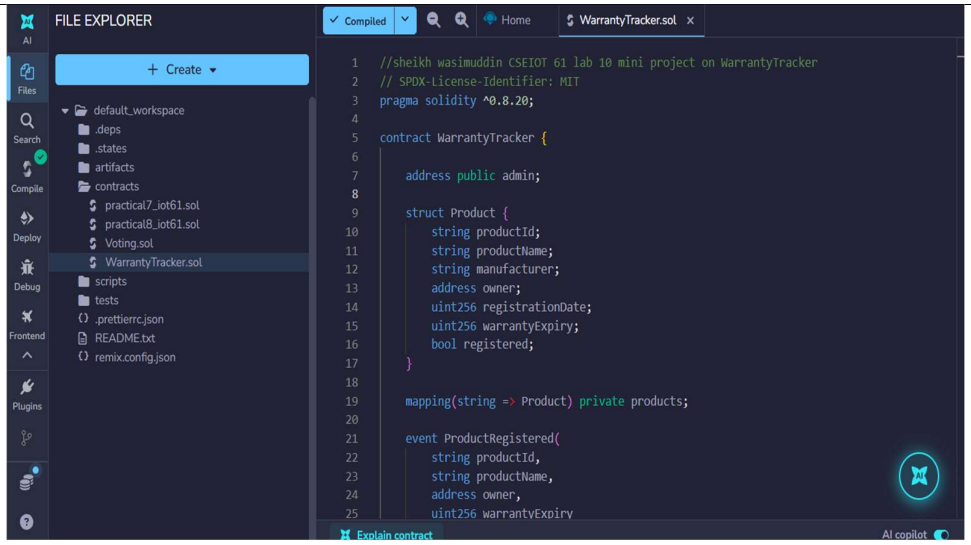
	interacting with the application. Using **ethers.js**, the frontend communicates with the deployed smart contract and allows users to register products, check warranty validity, view ownership information, and transfer ownership. Since the main records are maintained on the blockchain rather than a centralized database, the system provides a secure, transparent, and tamper-resistant method for warranty management. Overall, WarrantyChain demonstrates how blockchain and smart contracts can be applied to real-world product management. It improves trust between customers, manufacturers, and sellers by providing verifiable warranty records and a transparent ownership history.
Procedure and Execution (100 Words)	1. **Set up MetaMask** and create a wallet with the **Ethereum Sepolia Testnet** enabled. 2. **Obtain Sepolia test ETH** from a testnet faucet for transaction fees. 3. Open **Remix IDE** and create WarrantyTracker.sol. 4. Write the **Solidity smart contract** for product registration, warranty verification, and ownership transfer. 5. **Compile** the smart contract using Solidity Compiler in Remix. 6. Connect Remix to **MetaMask using Injected Provider** and select the **Sepolia network**. 7. **Deploy the smart contract** and copy the deployed contract address. 8. Create a **React + Vite frontend** and install ethers.js. 9. Add the deployed contract address and **ABI** to the React project. 10. Implement the frontend functions to **connect MetaMask, register products, verify warranties, and transfer ownership**. 11. Run the React application using npm run dev. 12. Connect the application to MetaMask and test all functions on Sepolia. 13. Verify transactions and the deployed contract using **Sepolia Etherscan**. 14. Record screenshots of the **Remix deployment, MetaMask transactions, frontend, and warranty verification** for the project documentation.
	Stepwise Screenshots:

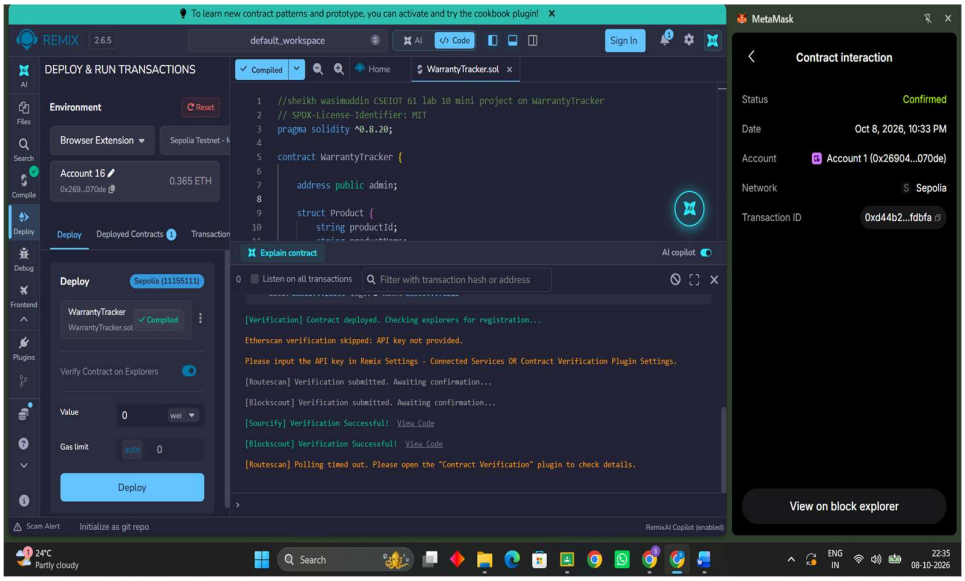
Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.





Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

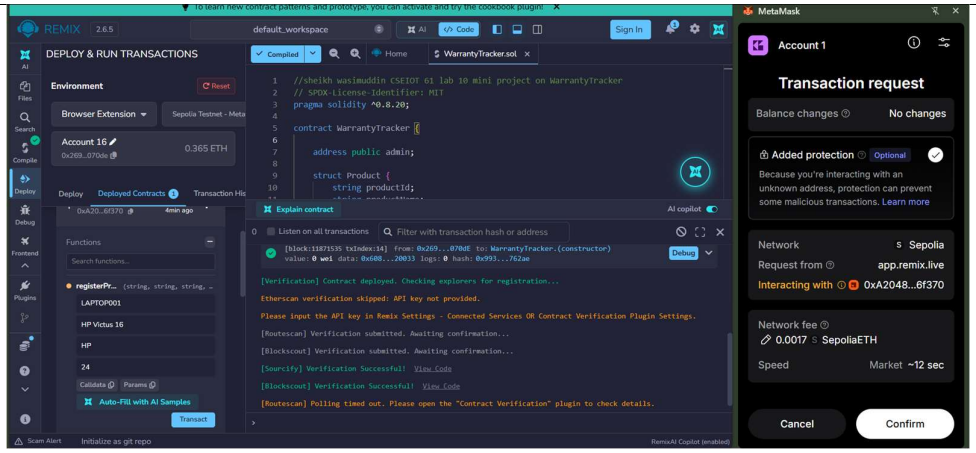
Department of Computer Science & Engineering (IoT)

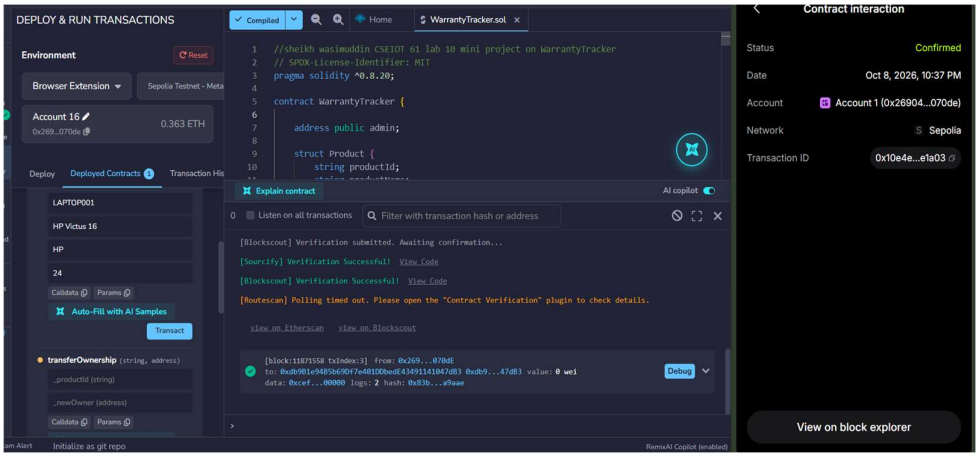
Vision of the Department

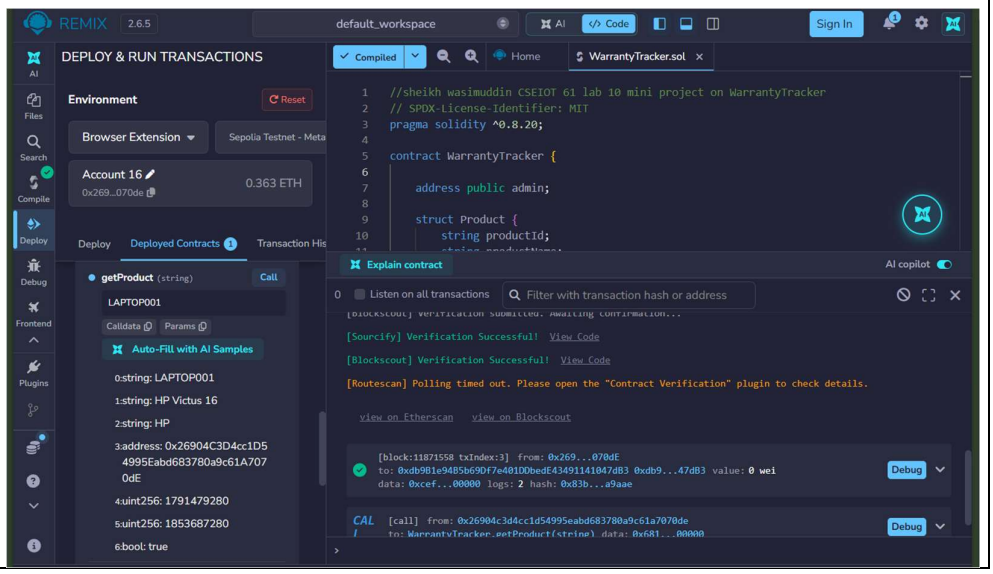
To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.





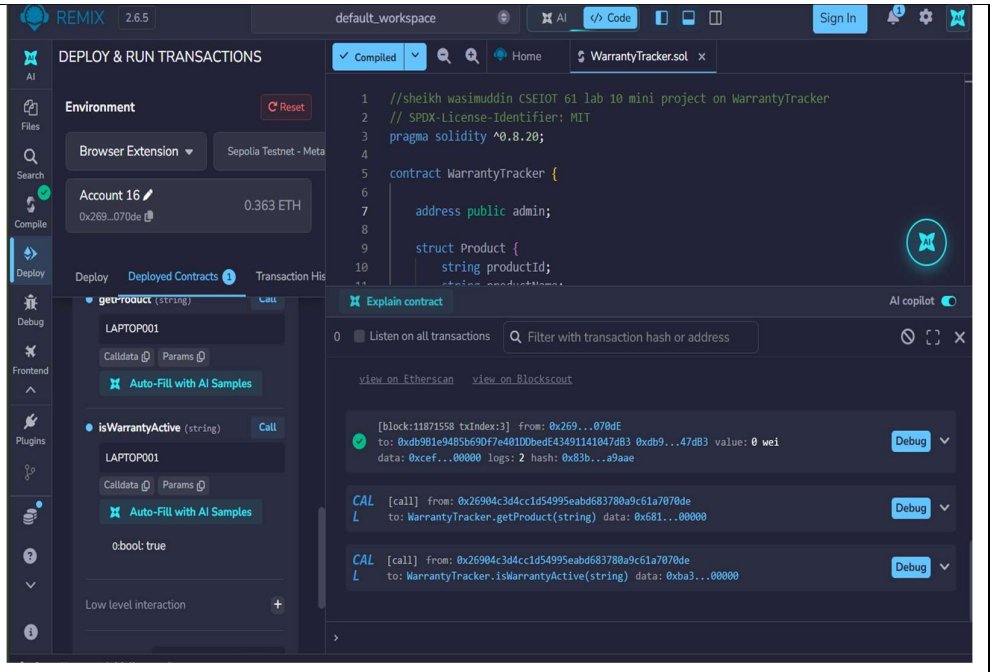


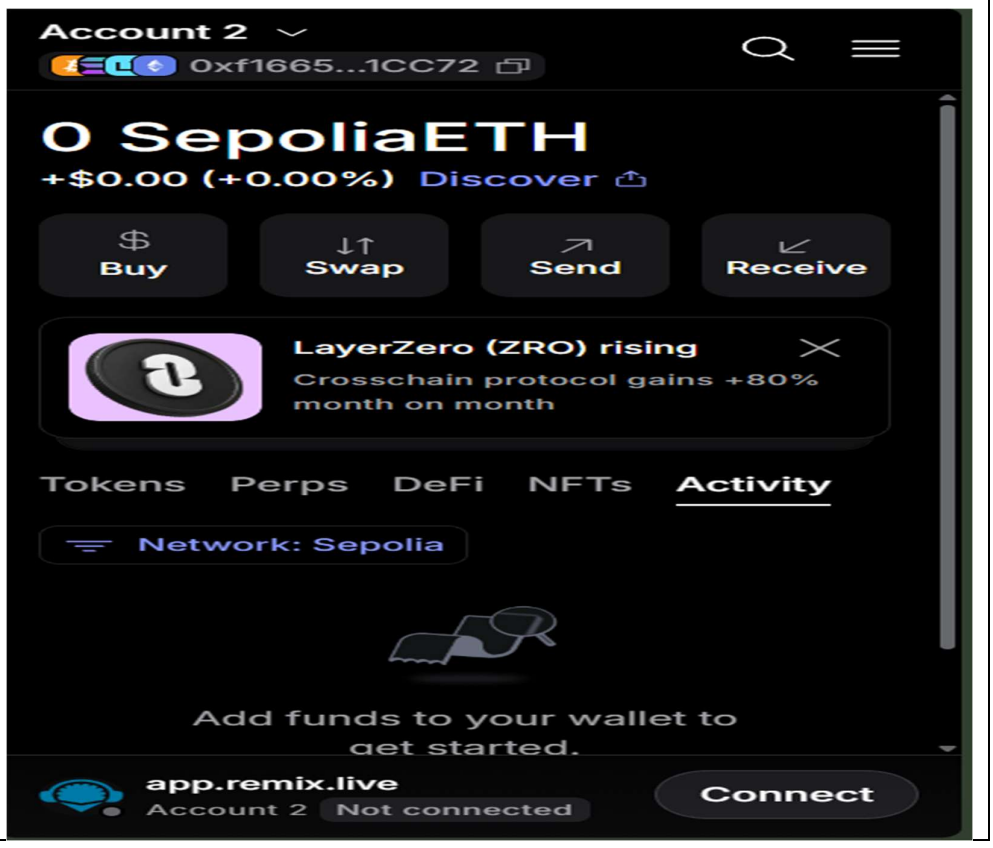
Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.



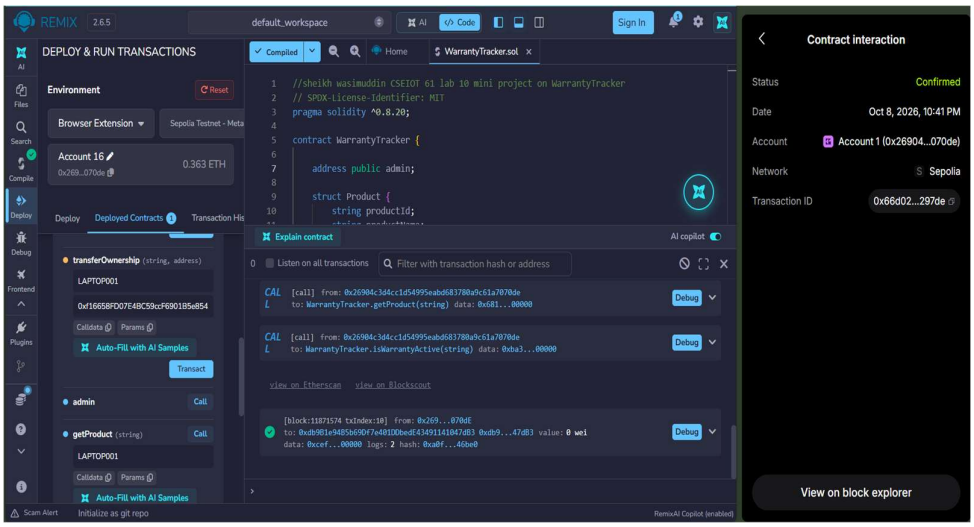


Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.



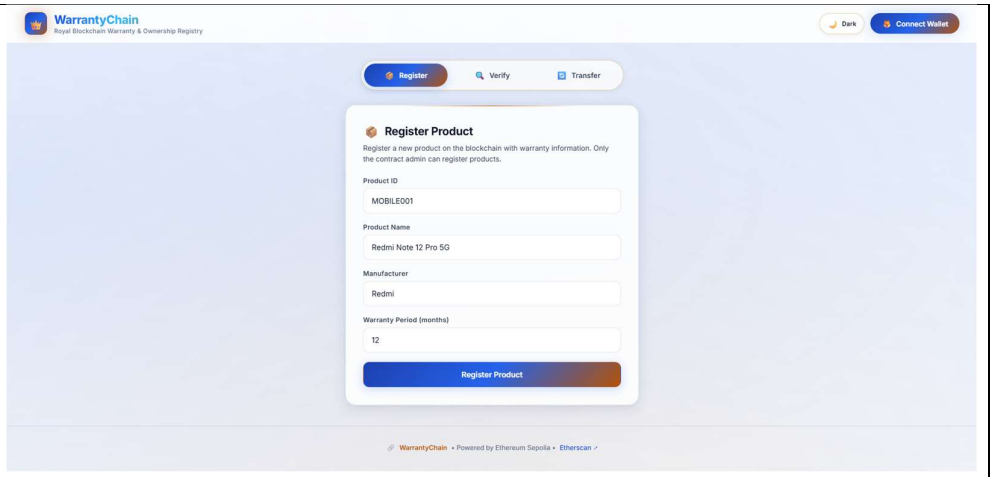


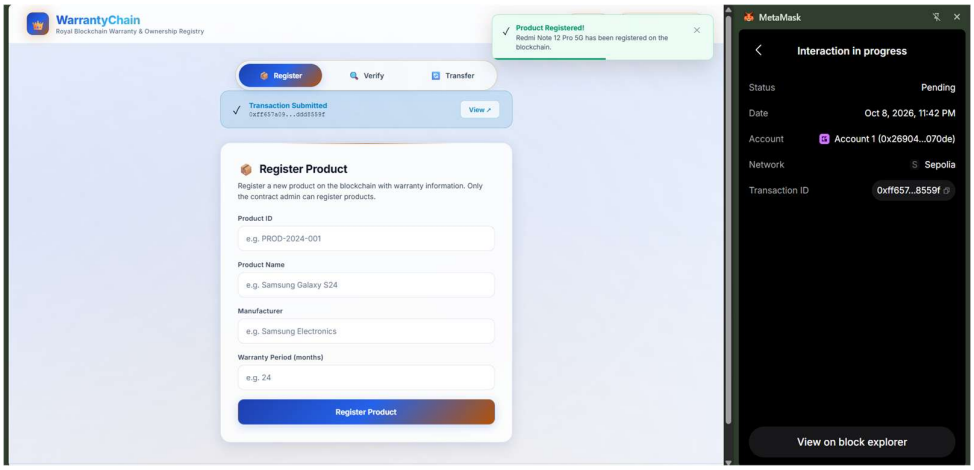
Vision of the Department

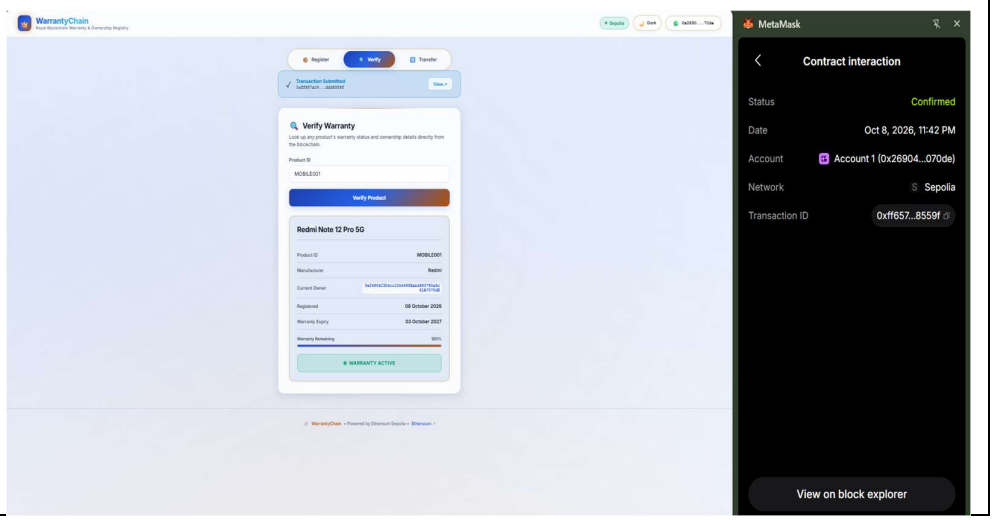
To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.





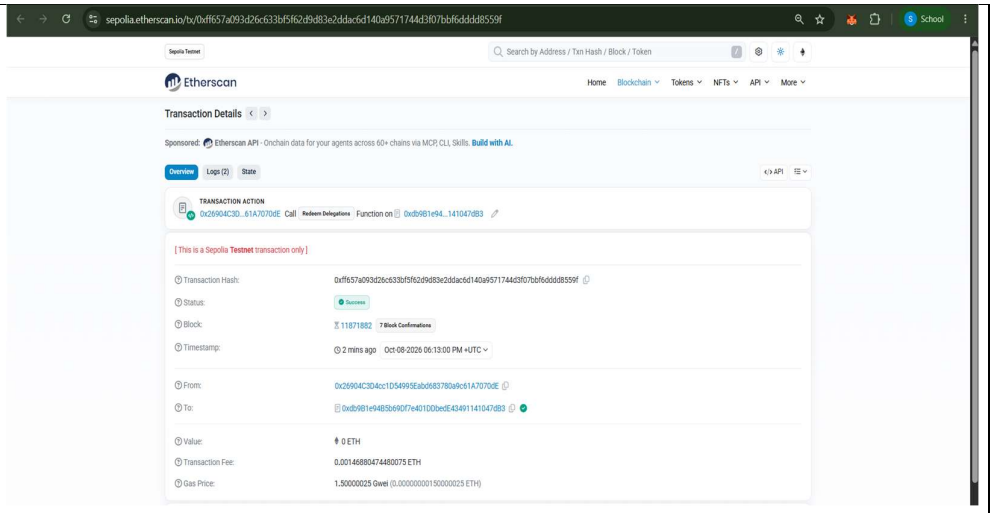


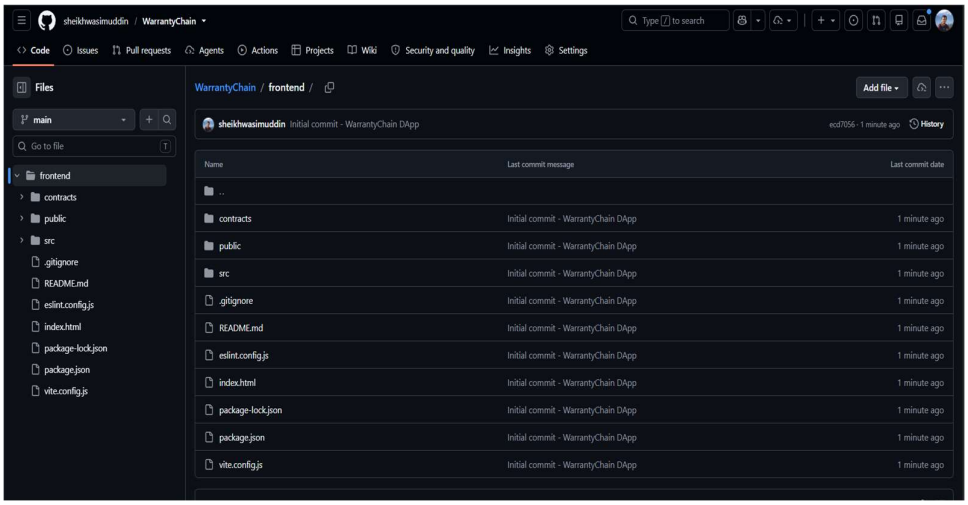
Vision of the Department

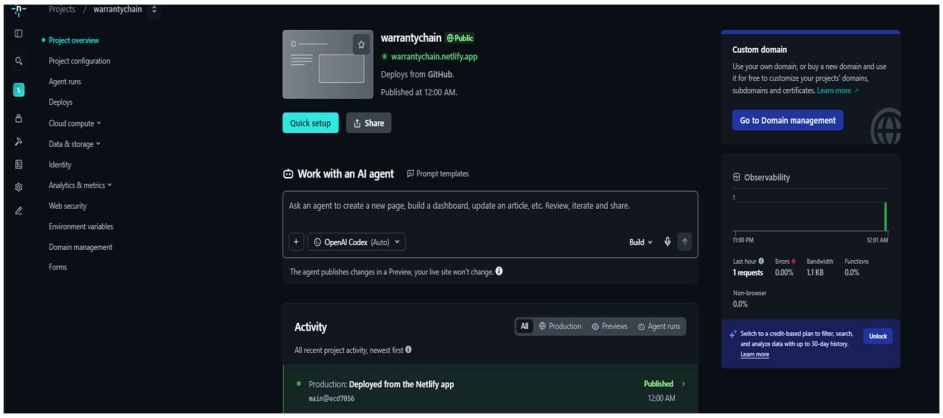
To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.







Code	***WarrantyTracker.sol***
	
	//sheikh wasimuddin CSEIOT 61 lab 10 mini project on WarrantyTracker
	// SPDX-License-Identifier: MIT
	pragma solidity ^0.8.20;
	
	contract WarrantyTracker {
	
	address public admin;
	
	struct Product {
	string productId;
	string productName;
	string manufacturer;
	address owner;
	uint256 registrationDate;
	uint256 warrantyExpiry;
	bool registered;
	}
	
	mapping(string => Product) private products;
	
	event ProductRegistered(
	string productId,
	string productName,
	address owner,
	uint256 warrantyExpiry
	);
	
	event OwnershipTransferred(
	string productId,
	address oldOwner,
	address newOwner
	);
	
	constructor() {

```
    admin = msg.sender;
}

modifier onlyAdmin() {
    require(
        msg.sender == admin,
        "Only admin can perform this action"
    );
    _;
}

function registerProduct(
    string memory _productId,
    string memory _productName,
    string memory _manufacturer,
    uint256 _warrantyMonths
) public onlyAdmin {

    require(
        !products[_productId].registered,
        "Product already registered"
    );

    require(
        _warrantyMonths > 0,
        "Warranty must be greater than zero"
    );

    uint256 expiry =
        block.timestamp +
        (_warrantyMonths * 30 days);

    products[_productId] = Product({
        productId: _productId,
        productName: _productName,
        manufacturer: _manufacturer,
        owner: msg.sender,
        registrationDate: block.timestamp,
        warrantyExpiry: expiry,
        registered: true
    });

    emit ProductRegistered(
        _productId,
```

```
        _productName,
        msg.sender,
        expiry
    );
}

function getProduct(
    string memory _productId
)
    public
    view
    returns (
        string memory,
        string memory,
        string memory,
        address,
        uint256,
        uint256,
        bool
    )
{
    require(
        products[_productId].registered,
        "Product not found"
    );
    Product memory p = products[_productId];
    return (
        p.productId,
        p.productName,
        p.manufacturer,
        p.owner,
        p.registrationDate,
        p.warrantyExpiry,
        p.registered
    );
}

function isWarrantyActive(
    string memory _productId
)
    public
    view
    returns (bool)
{
```

```
        require(
            products[_productId].registered,
            "Product not found"
        );

        return
            block.timestamp <
            products[_productId].warrantyExpiry;
    }

    function transferOwnership(
        string memory _productId,
        address _newOwner
    )
        public
    {
        require(
            products[_productId].registered,
            "Product not found"
        );

        require(
            products[_productId].owner == msg.sender,
            "Only current owner can transfer"
        );

        require(
            _newOwner != address(0),
            "Invalid new owner"
        );

        address oldOwner =
            products[_productId].owner;

        products[_productId].owner =
            _newOwner;

        emit OwnershipTransferred(
            _productId,
            oldOwner,
            _newOwner
        );
    }
}
```

Contract.js

```
export const contractAddress =
  "0xA20483F1d45FE61D3639299e3e5ab5C8C076f370";

export const contractABI = [
  {
    "inputs": [],
    "stateMutability": "nonpayable",
    "type": "constructor"
  },
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": false,
        "internalType": "string",
        "name": "productId",
        "type": "string"
      },
      {
        "indexed": false,
        "internalType": "address",
        "name": "oldOwner",
        "type": "address"
      },
      {
        "indexed": false,
        "internalType": "address",
        "name": "newOwner",
        "type": "address"
      }
    ],
    "name": "OwnershipTransferred",
    "type": "event"
  },
  {
    "anonymous": false,
    "inputs": [
      {
        "indexed": false,
        "internalType": "string",
        "name": "productId",
        "type": "string"
```

```
    },
    {
     "indexed": false,
     "internalType": "string",
     "name": "productName",
     "type": "string"
    },
    {
     "indexed": false,
     "internalType": "address",
     "name": "owner",
     "type": "address"
    },
    {
     "indexed": false,
     "internalType": "uint256",
     "name": "warrantyExpiry",
     "type": "uint256"
    }
   ],
   "name": "ProductRegistered",
   "type": "event"
  },
  {
   "inputs": [
    {
     "internalType": "string",
     "name": "_productId",
     "type": "string"
    },
    {
     "internalType": "string",
     "name": "_productName",
     "type": "string"
    },
    {
     "internalType": "string",
     "name": "_manufacturer",
     "type": "string"
    },
    {
     "internalType": "uint256",
     "name": "_warrantyMonths",
     "type": "uint256"
```

```
    }
   ],
   "name": "registerProduct",
   "outputs": [],
   "stateMutability": "nonpayable",
   "type": "function"
  },
  {
   "inputs": [
    {
     "internalType": "string",
     "name": "_productId",
     "type": "string"
    },
    {
     "internalType": "address",
     "name": "_newOwner",
     "type": "address"
    }
   ],
   "name": "transferOwnership",
   "outputs": [],
   "stateMutability": "nonpayable",
   "type": "function"
  },
  {
   "inputs": [],
   "name": "admin",
   "outputs": [
    {
     "internalType": "address",
     "name": "",
     "type": "address"
    }
   ],
   "stateMutability": "view",
   "type": "function"
  },
  {
   "inputs": [
    {
     "internalType": "string",
     "name": "_productId",
     "type": "string"
```

```
    }
   ],
   "name": "getProduct",
   "outputs": [
    {
     "internalType": "string",
     "name": "",
     "type": "string"
    },
    {
     "internalType": "string",
     "name": "",
     "type": "string"
    },
    {
     "internalType": "string",
     "name": "",
     "type": "string"
    },
    {
     "internalType": "address",
     "name": "",
     "type": "address"
    },
    {
     "internalType": "uint256",
     "name": "",
     "type": "uint256"
    },
    {
     "internalType": "uint256",
     "name": "",
     "type": "uint256"
    },
    {
     "internalType": "bool",
     "name": "",
     "type": "bool"
    }
   ],
   "stateMutability": "view",
   "type": "function"
  },
  {
```

	"inputs": [
	{
	"internalType": "string",
	"name": "_productId",
	"type": "string"
	}
	],
	"name": "isWarrantyActive",
	"outputs": [
	{
	"internalType": "bool",
	"name": "",
	"type": "bool"
	}
	],
	"stateMutability": "view",
	"type": "function"
	}
	];

Link	**https://warrantychain.netlify.app/**
Conclusion	The **WarrantyChain** project successfully demonstrates how blockchain technology can be used to create a secure, transparent, and tamper-resistant product warranty and ownership management system. By implementing the system using **Solidity, Ethereum Sepolia Testnet, MetaMask, React, Vite, and ethers.js**, product details, warranty validity, and ownership information can be recorded and verified on-chain. Overall, **WarrantyChain demonstrates the practical application of blockchain and smart contracts in real-world product lifecycle management**, with potential for future improvements such as **QR-code-based verification, manufacturer dashboards, transaction history, NFT-based warranties, and integration with real-world IoT devices**.
Date	06/10/2026